

Generation

Today's Agenda

Decoding techniques

Efficient decoding

More practice problems

Decoding Techniques

Once upon a time...

You're setting up **Qwen 3.8 Flash Next** with an API:

```
chat_response = client.chat.completions.create(
    model="Qwen/Qwen3.8-Flash-Next",
    messages=messages,
    extra_body={
        "chat_template_kwargs": {"enable_thinking": False},
    },
)
print("Chat response:", chat_response)
```

However, this API call also has additional hyperparameters you could set...

Once upon a time...

You read, on the model's Hugging Face model card:

Sampling Parameters

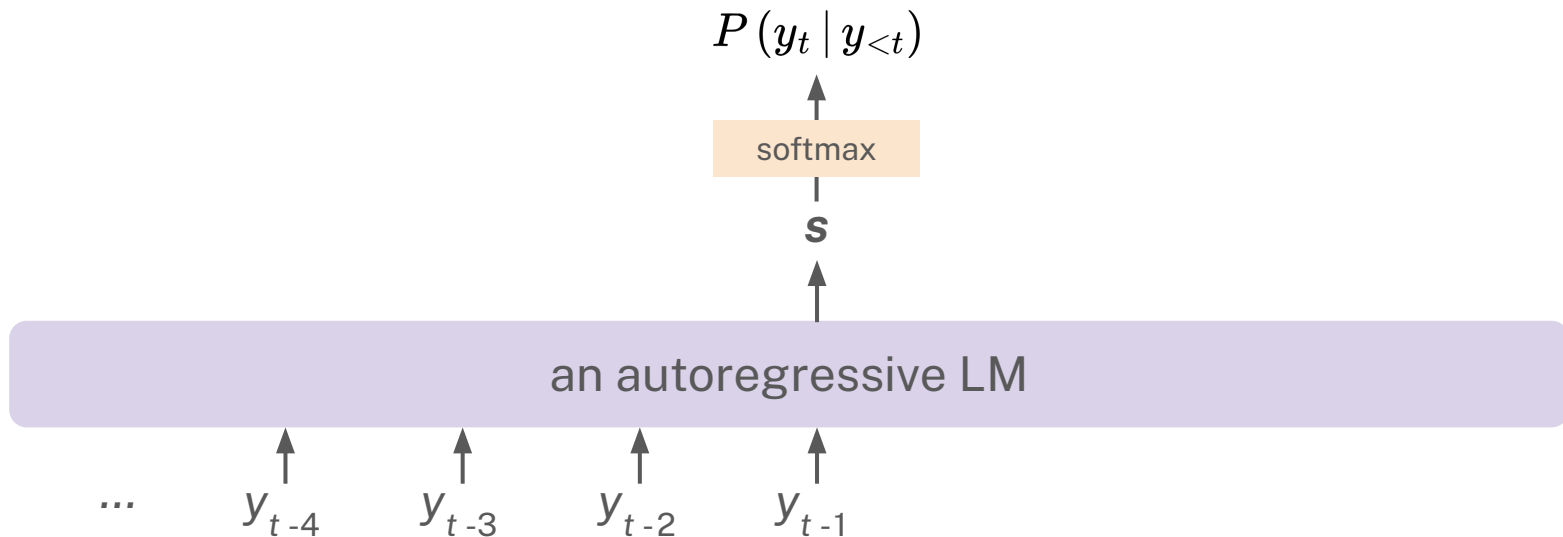
We suggest using the following sets of sampling parameters:

`temperature=0.7, top_p=0.80, top_k=20, min_p=0.0,
presence_penalty=1.5, repetition_penalty=1.0`

Today, we'll learn what these parameters mean for generation, and more!

Remember...

At each time step t , an LM computes a vector of scores $\mathbf{s} \in \mathbb{R}^{|V|}$, and we compute the prob distribution P over our vocab V using these scores:



Remember...

At each time step t , an LM computes a vector of scores $\mathbf{s} \in \mathbb{R}^{|V|}$, and we compute the prob distribution P over our vocab V using these scores.

Each index of P corresponds to a token w in our vocab:

$$P(y_t = w \mid y_{<t}) = \frac{\exp(s_w)}{\sum_{w' \in V} \exp(s_{w'})} \quad \mathbf{s}_w = \text{score for token } w$$

A **decoding algorithm** defines a function to select a token from this distribution:

$$\hat{y}_t = g(P(y_t \mid y_{<t}))$$

As we generate, we append each $\hat{y}_t$ to our growing output sequence.

Deterministic decoding

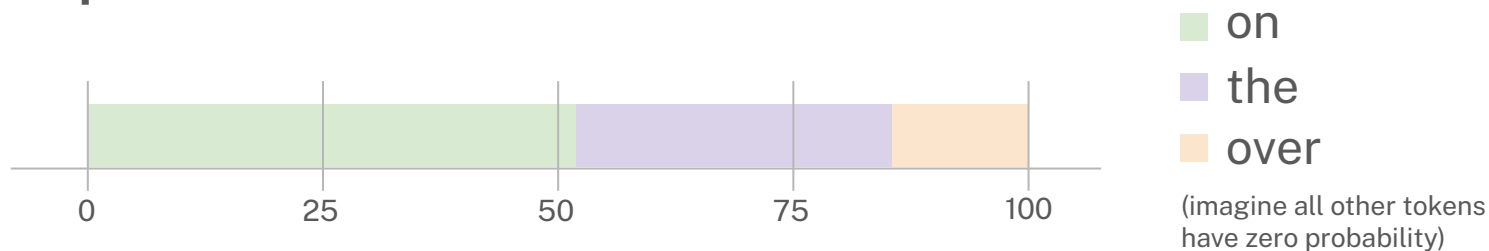
Greedy decoding

Just choose the most probable token at each generation step, without any random sampling.

$$\hat{y}_t = \arg \max_{w \in V} (P(y_t = w \mid y_{<t}))$$

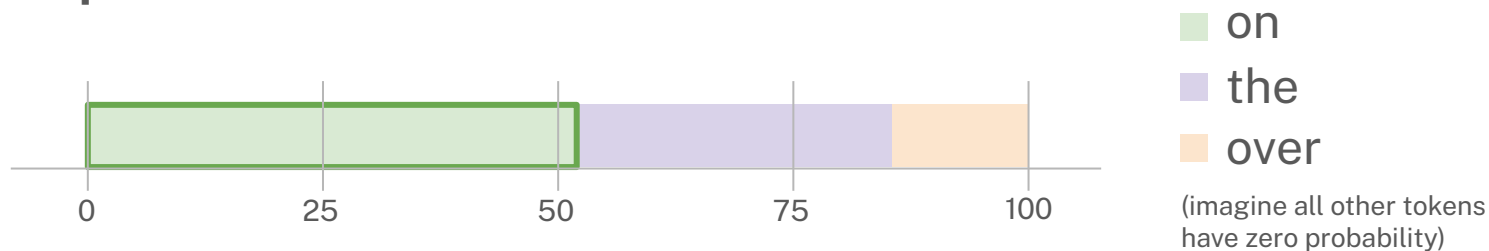
Greedy decoding

Output sequence: ??



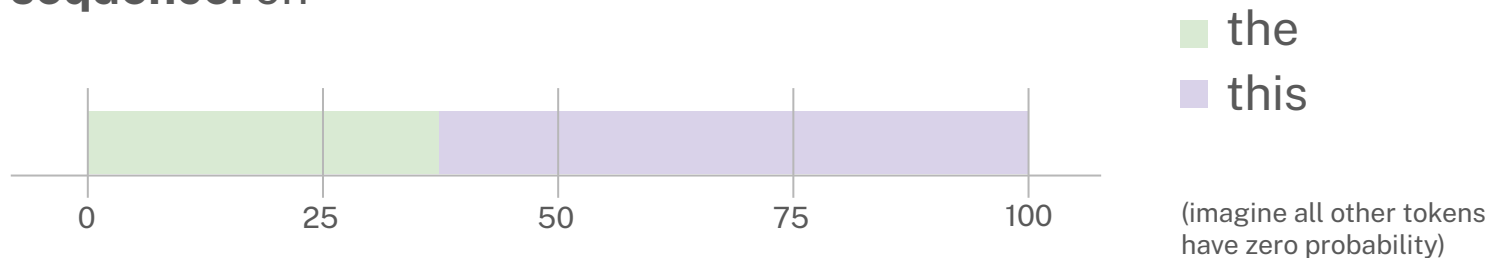
Greedy decoding

Output sequence: on



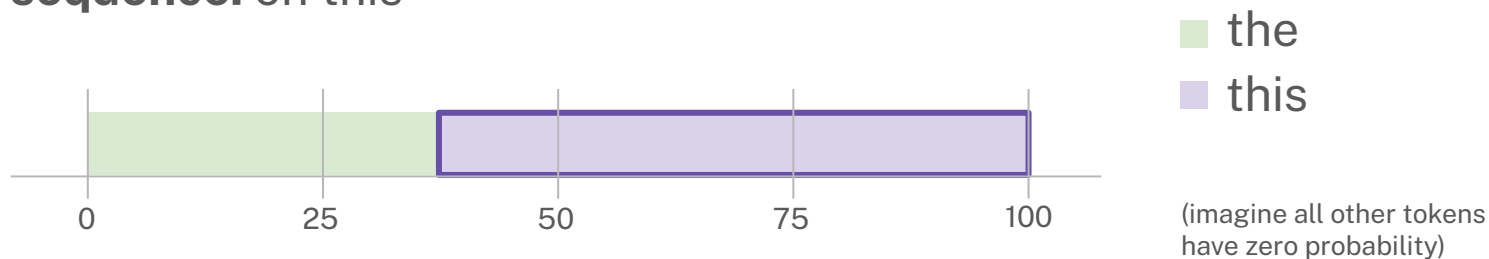
Greedy decoding

Output sequence: on



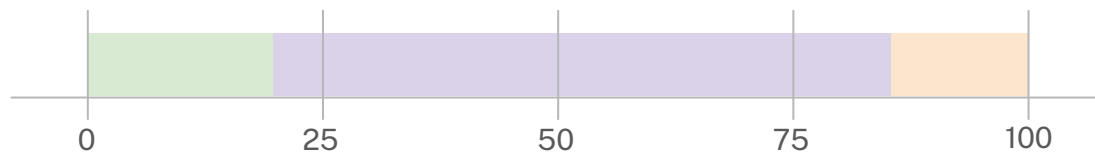
Greedy decoding

Output sequence: on this



Greedy decoding

Output sequence: on this

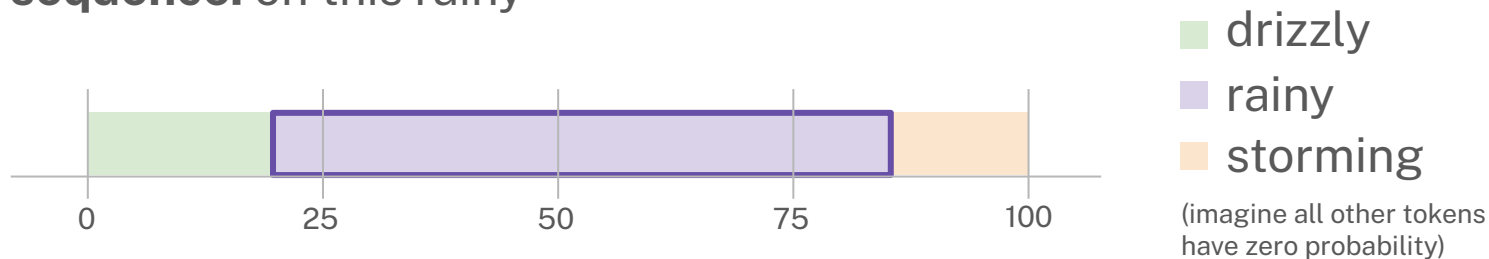


■ drizzly
■ rainy
■ storming

(imagine all other tokens
have zero probability)

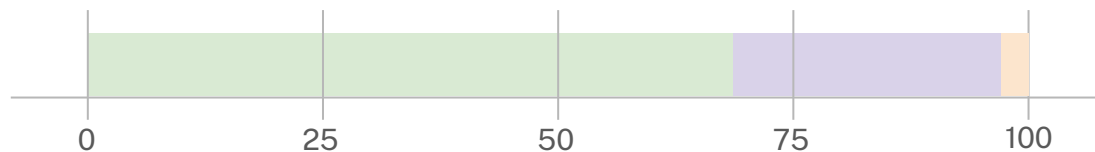
Greedy decoding

Output sequence: on this rainy



Greedy decoding

Output sequence: on this rainy



■ evening

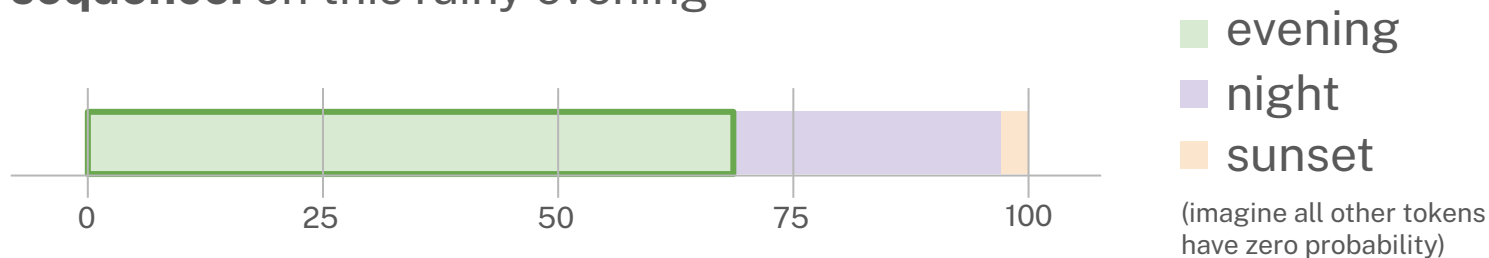
■ night

■ sunset

(imagine all other tokens
have zero probability)

Greedy decoding

Output sequence: on this rainy evening



Greedy decoding

Just choose the most probable token at each generation step, without any random sampling.

$$\hat{y}_t = \arg \max_{w \in V} (P(y_t = w \mid y_{<t}))$$

Limitation: we actually care about the prob of *the entire output sequence*!

Greedy isn't guaranteed to give us the most probable sequence as a whole, just at the current time step t .

A token that looks slightly worse *right now* might lead us to future tokens that altogether are better, but we don't venture down those paths.

Deterministic decoding

Beam search

A better approx for the argmax of the prob of the entire sequence.

During generation, maintain a “beam” of n sequences

At each generation step i :

- Select the n most likely next tokens, and create n more sequences.

- Look at all n^2 sequences generated so far, and discard all but the n most likely sequences.

At the end, select the sequence with the highest prob among the set.

Beam search, $n = 3$

$t = 1$

the

0.4

in

0.3

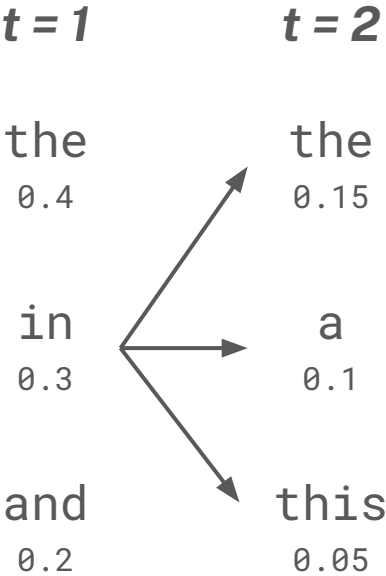
and

0.2

Here are our initial set of three beams.

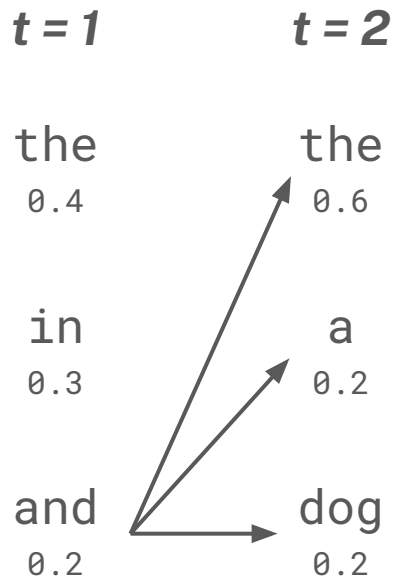
At each time step, we only look at the top n (three) most probable next tokens.

Beam search, $n = 3$



Current candidate	Probability
the list	0.4×0.3
the look	0.3×0.2
the parents	0.2×0.1
in the	0.3×0.15
in a	0.3×0.1
in this	0.3×0.05

Beam search, $n = 3$



Current candidate	Probability
the list	0.4×0.3
the look	0.3×0.2
the parents	0.2×0.1
in the	0.3×0.15
in a	0.3×0.1
in this	0.3×0.05
and the	0.2×0.6
and a	0.2×0.2
and dog	0.2×0.2

Beam search, $n = 3$

We have n^2 (a.k.a. nine) partial sequences generated here!

Current candidate	Probability
the list	0.12
the look	0.06
the parents	0.02
in the	0.05
in a	0.03
in this	0.02
and the	0.12
and a	0.04
and dog	0.04

Beam search, $n = 3$

Keep only the top $n = 3$ partial sequences as new “beams” for the next iteration.

Current candidate	Probability
the list	0.12
the look	0.06
the parents	0.02
in the	0.05
in a	0.03
in this	0.02
and the	0.12
and a	0.04
and dog	0.04

Beam search, $n = 3$

the list

0.12

the look

0.06

and the

0.12

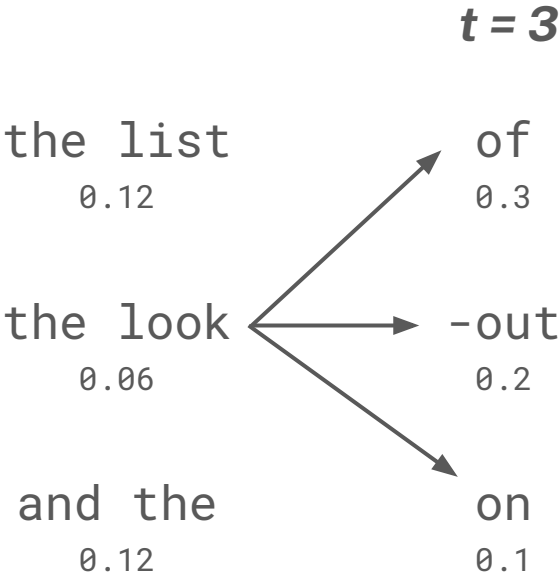
[illegible]

Beam search, $n = 3$

$t = 3$

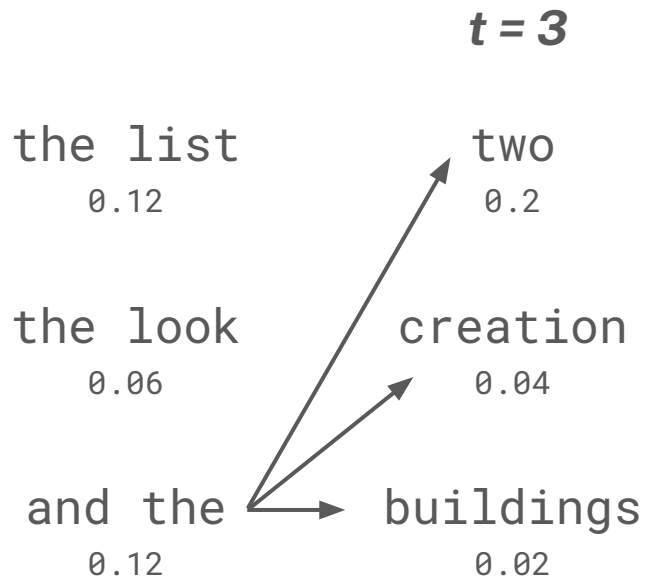
[illegible]

Beam search, $n = 3$



Current candidate	Probability
the list of	0.004
the list goes	0.002
the list is	0.002
the look of	0.018
the lookout	0.012
the look on	0.006

Beam search, $n = 3$



Current candidate	Probability
the list of	0.004
the list goes	0.002
the list is	0.002
the look of	0.018
the lookout	0.012
the look on	0.006
and the two	0.024
and the creation	0.005
and the buildings	0.002

Beam search, $n = 3$

the list

0.12

the look

0.06

and the

0.12

Current candidate	Probability
the list of	0.004
the list goes	0.002
the list is	0.002
the look of	0.018
the lookout	0.012
the look on	0.006
and the two	0.024
and the creation	0.005
and the buildings	0.002

Beam search, $n = 3$

When do we stop?

When all of the items in the beam have end with a special stop token, or when we've reached max sequence length.

Current candidate	Probability
the list of	0.004
the list goes	0.002
the list is	0.002
the look of	0.018
the lookout	0.012
the look on	0.006
and the two	0.024
and the creation	0.005
and the buildings	0.002

Beam search, $n = 3$

When do we stop?

When all of the items in the beam have end with a special stop token, or when we've reached max sequence length.

If we pretend we're done now, we'd select the sequence with the overall highest prob (normalized by length).

Current candidate	Probability
the list of	0.004
the list goes	0.002
the list is	0.002
the look of	0.018
the lookout	0.012
the look on	0.006
and the two	0.024
and the creation	0.005
and the buildings	0.002

Beam search

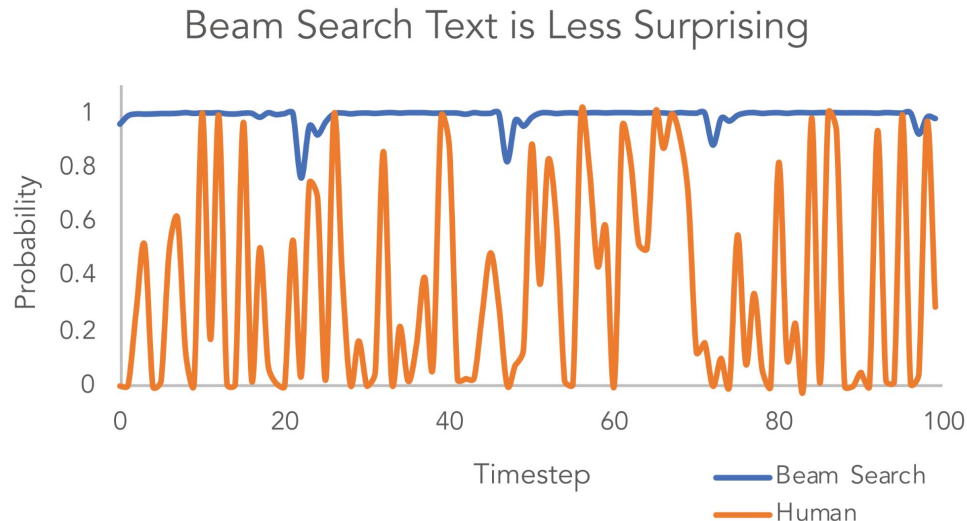
With $n = 1$, beam search reduces to greedy decoding.

It offers more output diversity than greedy decoding, and was the defacto standard in classical NLP models (e.g. seq2seq RNNs).

But it's no longer used in modern LMs!

Beam search

Approximating the argmax most probable sequence produces **repetitive and overall too-probable sequences**. Still not enough diversity, and loopy behavior.



...to provide an overview of the current state-of-the-art in the field of computer vision and machine learning, and to provide an overview of the current state-of-the-art in the field of computer vision and machine learning, and to provide an overview of the current state-of-the-art in the field of computer vision and machine learning, and to provide an overview of the current state-of-the-art in the field of computer vision and machine learning, and...

What if we inject some randomness?

Ancestral sampling

Let's sample directly from P , the probability distribution for the next word.

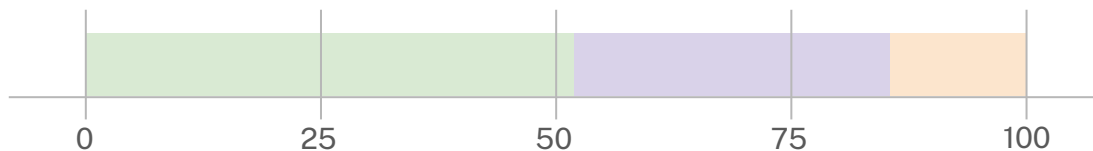
What if we inject some randomness?

Ancestral sampling

Let's sample directly from P , the probability distribution for the next word.

Output sequence: ??

Visually: the prob a word is chosen for the next step = how much of this bar from 0-100 it takes up.



■ on
■ the
■ over

(imagine all other tokens have zero probability)

What if we inject some randomness?

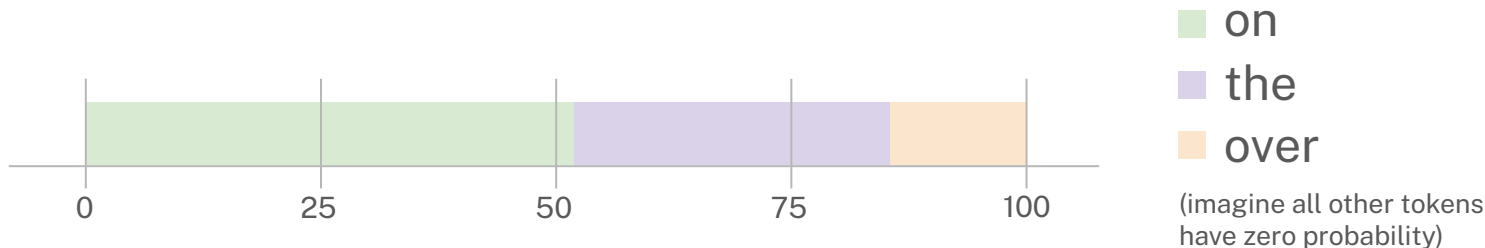
Ancestral sampling

Let's sample directly from P , the probability distribution for the next word.

Output sequence: ??

```
number = random(0, 100)
```

```
number ← 65
```

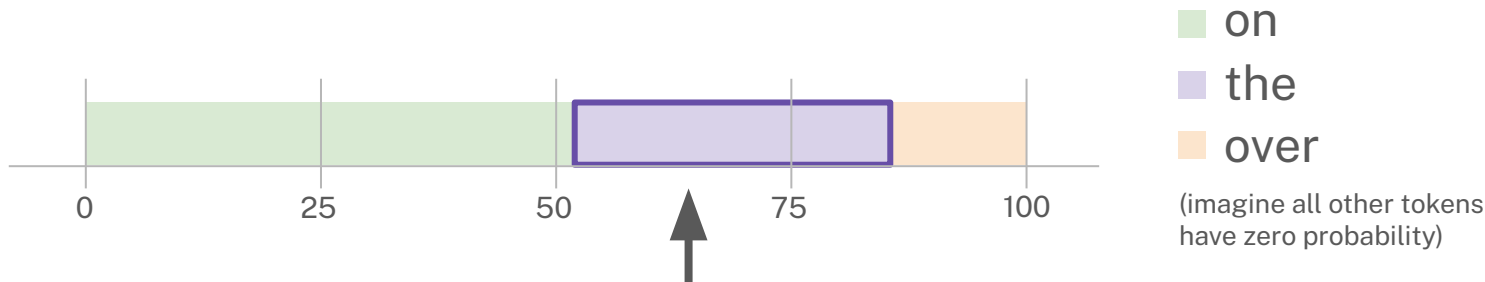


What if we inject some randomness?

Ancestral sampling

Let's sample directly from P , the probability distribution for the next word.

Output sequence: the
number = `random(0, 100)`
number $\leftarrow$ 65

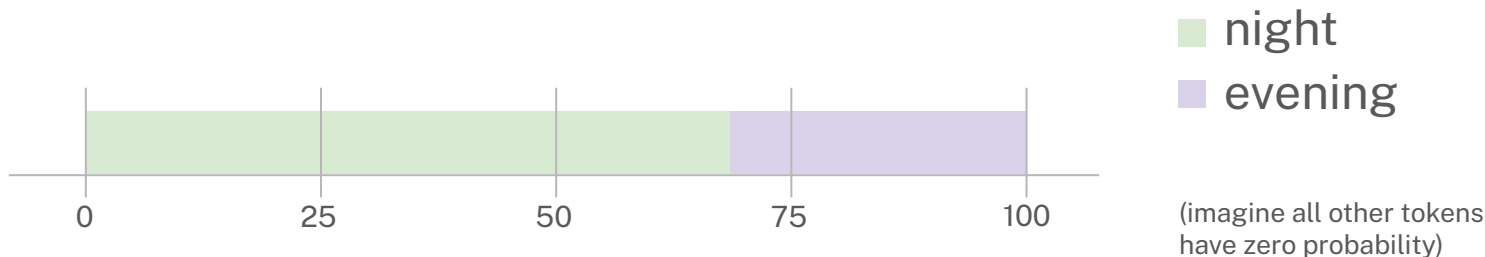


What if we inject some randomness?

Ancestral sampling

Let's sample directly from P , the probability distribution for the next word.

Output sequence: the
number = `random(0, 100)`
number $\leftarrow$ 26

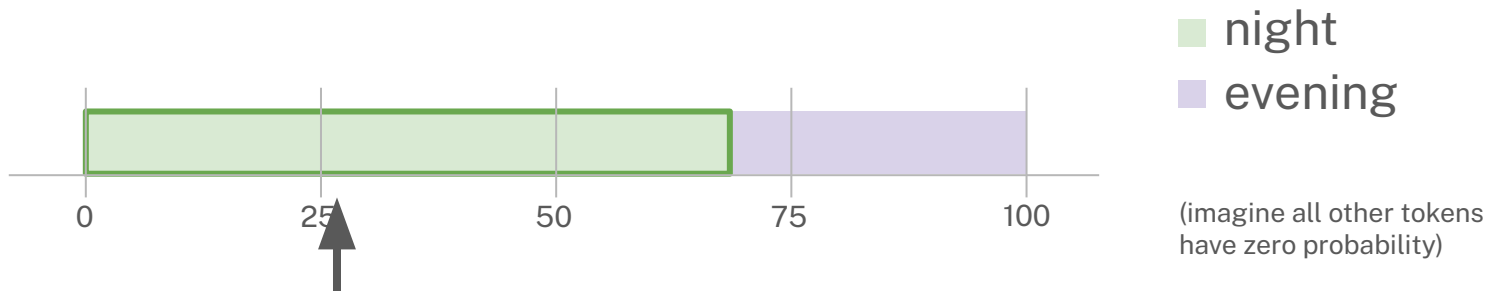


What if we inject some randomness?

Ancestral sampling

Let's sample directly from P , the probability distribution for the next word.

Output sequence: the night
number = random(0, 100)
number $\leftarrow$ 26

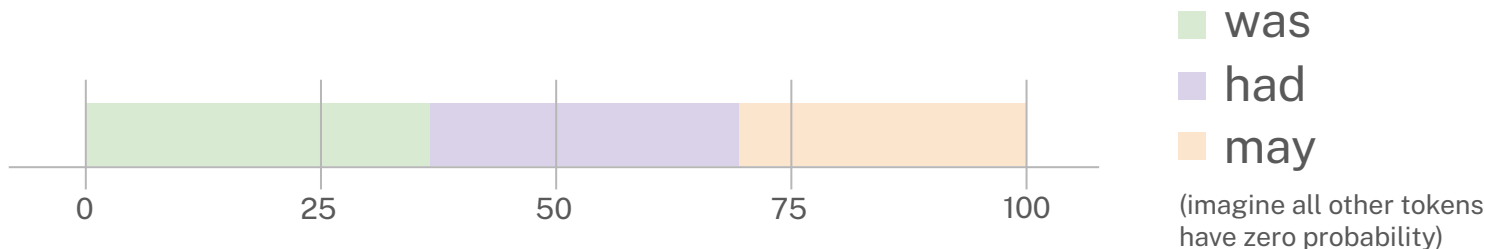


What if we inject some randomness?

Ancestral sampling

Let's sample directly from P , the probability distribution for the next word.

Output sequence: the night
number = random(0, 100)
number $\leftarrow$ 83



What if we inject some randomness?

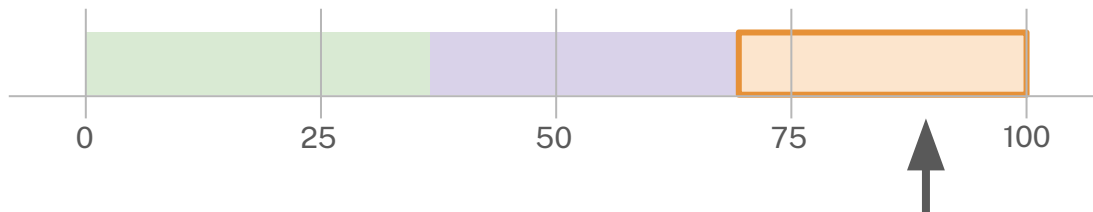
Ancestral sampling

Let's sample directly from P , the probability distribution for the next word.

Output sequence: the night may

```
number = random(0, 100)
```

```
number ← 83
```



and so on...

was

had

may

(imagine all other tokens
have zero probability)

Once upon a time...

You read, on Qwen 3.8 Flash Next's Hugging Face model card:

Sampling Parameters

We suggest using the following sets of sampling parameters:

temperature=0.7, top_p=0.80, top_k=20, min_p=0.0,
presence_penalty=1.5, repetition_penalty=1.0

Managing repetition

LMs like to loop and repeat themselves! Params implemented in APIs:

Presence penalty. Subtract a penalty based on whether an output token has already appeared: `logits -= presence_penalty * output_mask`

Repetition penalty. Penalize the model based on whether a token has already appeared. Scale a token's logit via multiplication or division:

```
seen = prompt_mask | output_mask
p = where(seen, repetition_penalty, 1.0)
logits *= where(logits > 0, 1/p, p)
```

Frequency penalty. Subtract a penalty based on the frequency of tokens that have already been generated: `logits -= frequency_penalty * output_counts`

Once upon a time...

You read, on Qwen 3.8 Flash Next's Hugging Face model card:

Sampling Parameters

We suggest using the following sets of sampling parameters:

temperature=0.7, top_p=0.80, top_k=20, min_p=0.0,
presence_penalty=1.5, repetition_penalty=1.0



Temperature

At each time step t , we compute the prob distribution P over our vocab V using a vector of scores:

$$P(y_t = w \mid y_{<t}) = \frac{\exp(s_w)}{\sum_{w' \in V} \exp(s_{w'})} \quad s_w = \text{score for token } w$$

What if we had a knob that could modify how “random” our sampling is?

Let's add a parameter that reshapes our probability distribution!

$$P(y_t = w \mid y_{<t}) = \frac{\exp(s_w / \tau)}{\sum_{w' \in V} \exp(s_{w'} / \tau)}$$



Temperature

This hyperparameter controls the “smoothness” of our distribution

$$P(y_t = w \mid y_{<t}) = \frac{\exp(s_w/\tau)}{\sum_{w' \in V} \exp(s_{w'}/\tau)}$$

$\tau = 1$: no changes to the probability distribution

Once upon a time, there
was a -----

little	
young	
king	
girl	
kingdom	



Temperature

This hyperparameter controls the “smoothness” of our distribution

$$P(y_t = w \mid y_{<t}) = \frac{\exp(s_w/\tau)}{\sum_{w' \in V} \exp(s_{w'}/\tau)}$$

$\tau < 1$: relative probability assigned to highest-prob token increases

Temperature is always non-negative.
Setting $\tau = 0$ recovers the argmax, and puts all of the mass on the highest probability token.

Once upon a time, there
was a _____

little	
young	
king	
girl	
kingdom	



Temperature

This hyperparameter controls the “smoothness” of our distribution

$$P(y_t = w \mid y_{<t}) = \frac{\exp(s_w/\tau)}{\sum_{w' \in V} \exp(s_{w'}/\tau)}$$

Recall how our distribution looks like
at $\tau = 1$...

Once upon a time, there
was a -----

little	
young	
king	
girl	
kingdom	



Temperature

This hyperparameter controls the “smoothness” of our distribution

$$P(y_t = w \mid y_{<t}) = \frac{\exp(s_w/\tau)}{\sum_{w' \in V} \exp(s_{w'}/\tau)}$$

$\tau > 1$: distribution becomes more uniform

Temperature allows us to control the entropy of the output distribution, **without changing the relative rankings of items** (unless we set $\tau = 0$ or $\tau = \infty$, and the latter is not possible in practice).

Once upon a time, there
was a -----

little	
young	
king	
girl	
kingdom	



Temperature

$\tau = 1$: no changes to the probability distribution

$\tau < 1$: distribution becomes more spiky and less diverse; probability is concentrated to the top tokens

$\tau > 1$: distribution becomes more uniform and more diverse; probability is spread across the vocabulary

Temperature is a **hyperparameter**, not a decoding algorithm by itself.

It can be applied to both beam search and sampling methods.

Once upon a time...

You read, on Qwen 3.8 Flash Next's Hugging Face model card:

Sampling Parameters

We suggest using the following sets of sampling parameters:

`temperature=0.7, top_p=0.80, top_k=20, min_p=0.0,
presence_penalty=1.5, repetition_penalty=1.0`

Issues

Beam search can degenerate, producing **repetitive and overall *too-probable* sequences**.

Sampling can become **incoherent**: the long tail includes lots of bad options that, on aggregate, still have a high chance of being sampled.



Top-k sampling

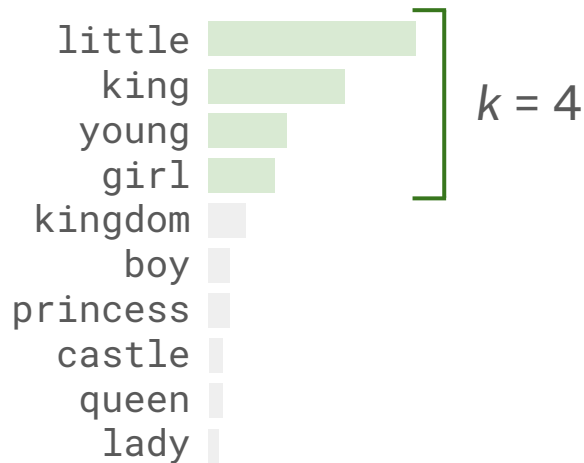
Let's trim off our long tail!

Identify the set of k tokens that have the highest probabilities

Set the probabilities of all but these tokens to 0

Renormalize by dividing the remaining probs by the sum of these k tokens' probs

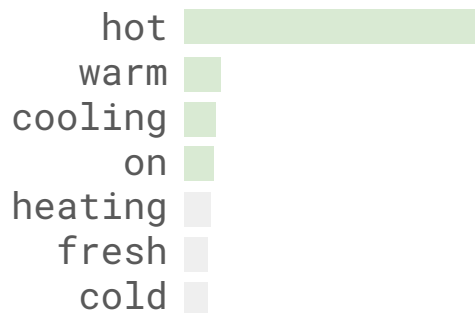
Once upon a time, there
was a _____



Top-k sampling



For a **flat** distribution, top- k sampling may cut off the tail too soon!



For a **peaked** distribution, it may cut off the tail too late!

Top-p sampling

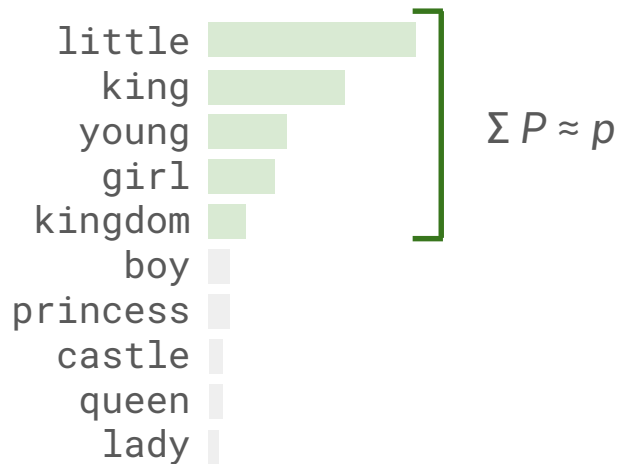
Also called **nucleus sampling**.

Add n tokens that have the highest probabilities to a set until their cumulative probability exceeds p

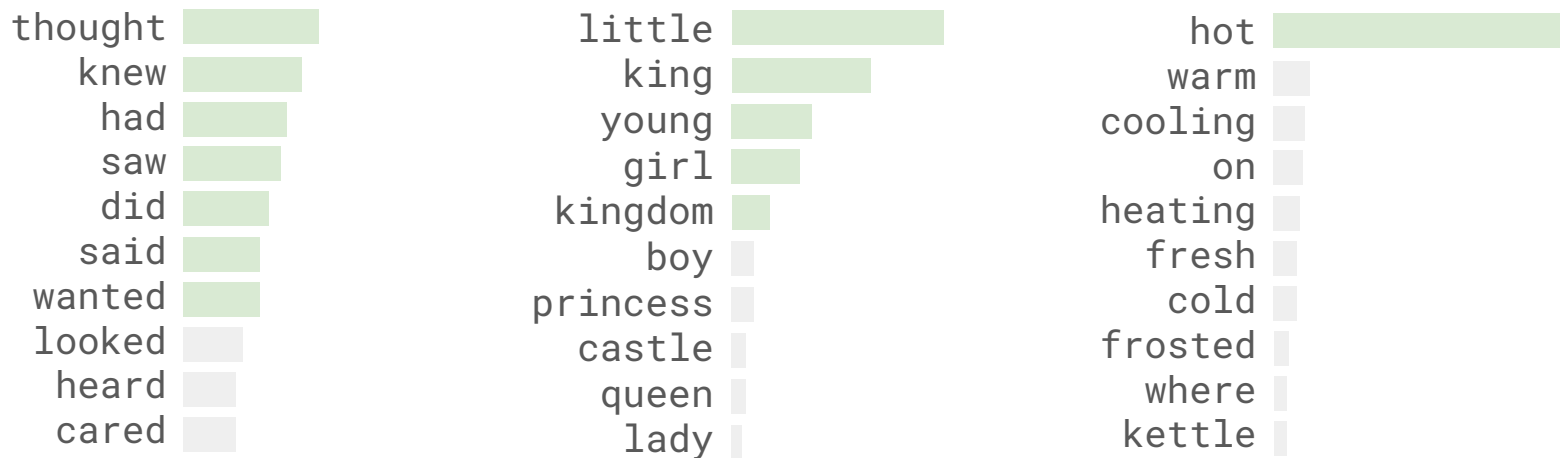
Set the probabilities of all but these n tokens to 0

Renormalize by dividing the remaining probs by the sum of these n tokens' probs

Once upon a time, there
was a -----



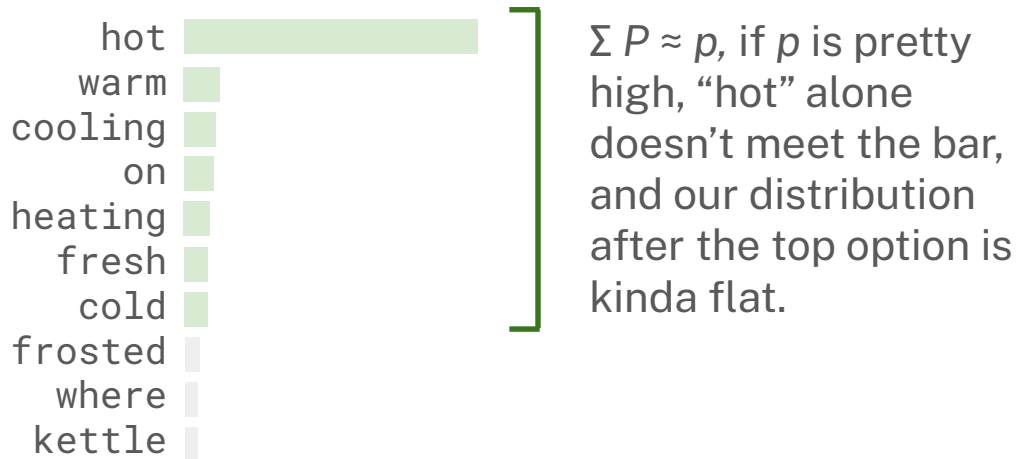
Top-p sampling



Sample from n tokens in the top p cumulative probability mass. Depending on the shape of the distribution, n varies!

Top-p sampling

With top-p sampling, sometimes low probability tokens still get included, because we're forced to meet our cumulative probability total of p :



Min-p sampling

Alternative: dynamically adjust the cut-off threshold based on the model's confidence.

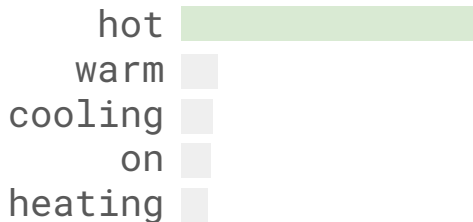
Identify the argmax token in the distribution, which has some probability $p_{\max}$.

Using a probability threshold min_p , we set $p_{\text{scaled}} = \text{min_p} \times p_{\max}$.

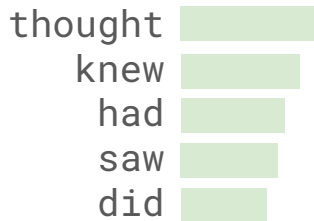
Set the probabilities of all tokens with prob less than p_{scaled} to 0.

Renormalize by dividing the remaining probs by their sum.

Focus on likeliest tokens when model is confident



Allow more diverse options when model is uncertain



Once upon a time...

You read, on Qwen 3.8 Flash Next's Hugging Face model card:

Sampling Parameters

We suggest using the following sets of sampling parameters:

temperature=0.7, top_p=0.80, top_k=20, min_p=0.0,
presence_penalty=1.5, repetition_penalty=1.0

Once upon a time...

You read, on Qwen 3.8 Flash Next's Hugging Face model card:

Sampling Parameters

We suggest using the following sets of sampling parameters:

`temperature=0.7, top_p=0.80, top_k=20, min_p=0.0,
presence_penalty=1.5, repetition_penalty=1.0`

We can modify hyperparameters like these based on whether our tasks have **a definite correct answer and reproducibility is needed**, versus when our tasks are more **open-ended and creative**.

Constrained decoding

Aside from cutting the long tail, we can also avoid “bad” tokens in other ways.

For some tasks, we know what tokens can or can't be next:

- maybe we can't generate more closing brackets than opening brackets.
- maybe we want to force outputs to adhere to json format.

Modern LMs fix these problems w/ scale, but weaker/smaller LMs can sometimes benefit from us constraining their output space with rules!

Given a set of possible continuations, set the probabilities of unwanted tokens to 0, and renormalize.

Tools: [xgrammar](#), [llguidance](#), [outlines](#)

Efficient Decoding

Processing a prompt

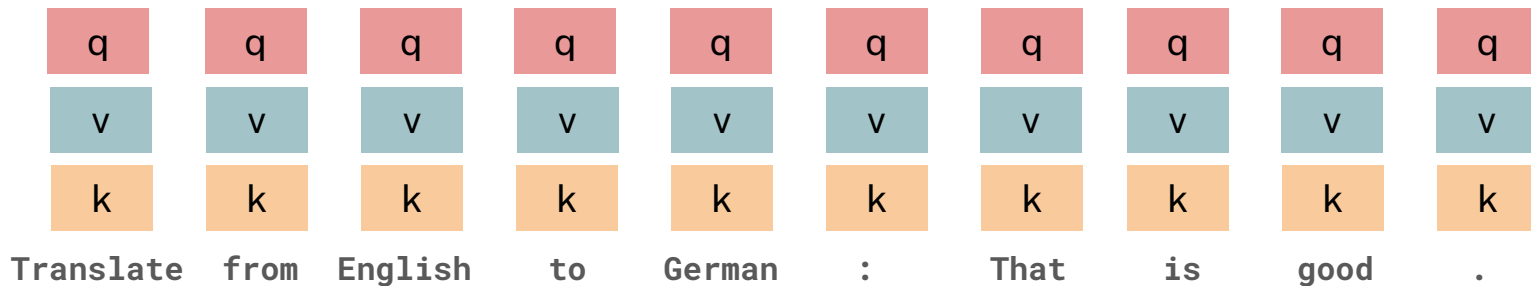
Let's say we have a prompt:

Translate from English to German: That is good.

Processing a prompt

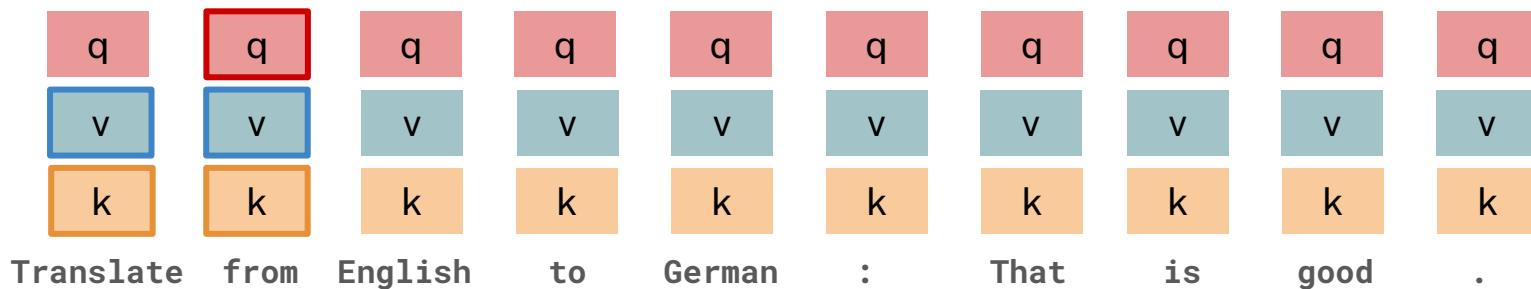
Pretend our tokenizer splits our prompt into ten tokens. Then, we have:

- 10 queries
- 10 keys
- 10 values



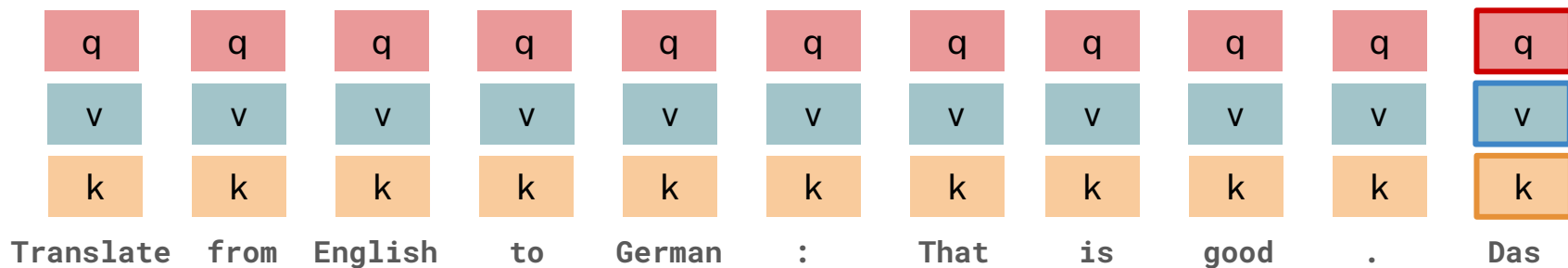
Processing a prompt

Remember, for masked self-attention, the query for “from” pays attention to (a.k.a. computes the dot product with) the keys for “Translate” and “from”, and computes a weight sum using those words’ values.



Processing a prompt

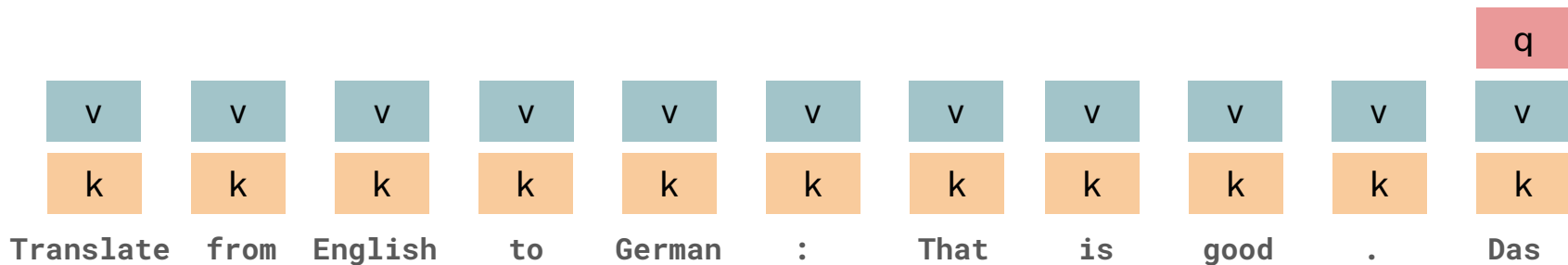
When we're generating the next word in our output, e.g. "Das", our model computes new query, key, and value vectors.



Processing a prompt

When we're generating the next word in our output, e.g. "Das":

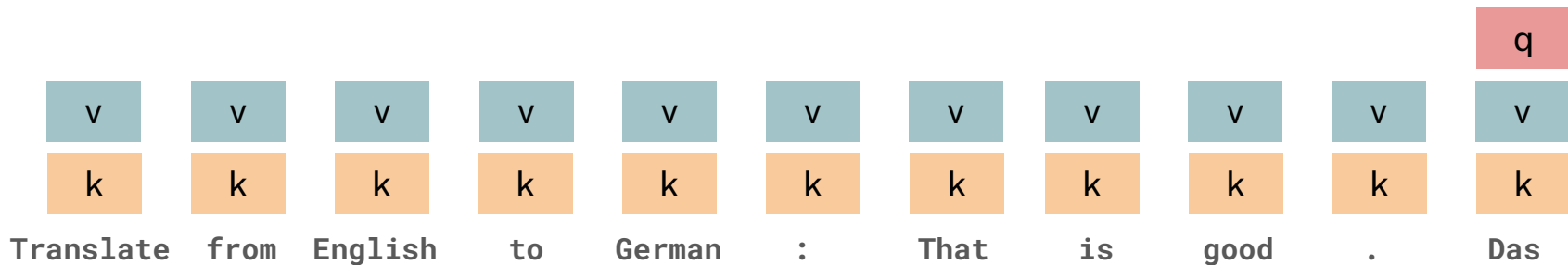
We don't need $q_1, \dots, q_{10}$ anymore! A query's only use is computing the output at its time step.



Processing a prompt

However, we still need $k_1, \dots, k_{10}$ and $v_1, \dots, v_{10}$!

Right now, q_{11} needs them to calculate self-attention, and they're reused by other future tokens as well.

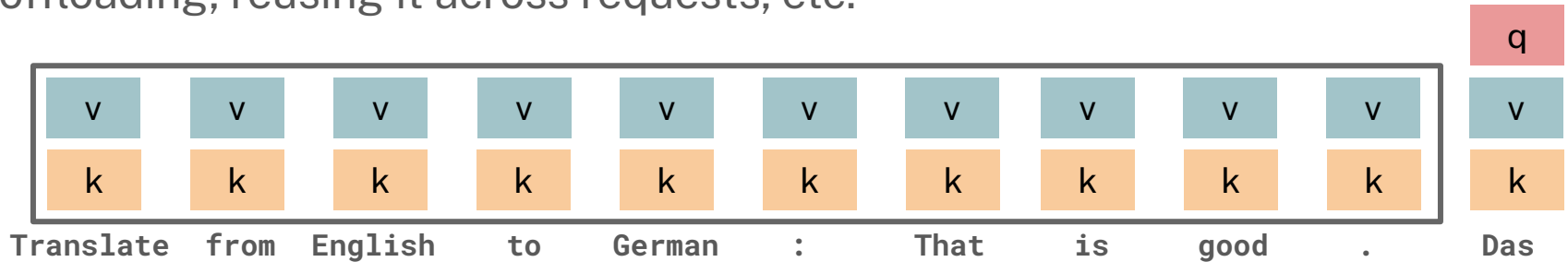


Processing a prompt

As a memory optimization trick, modern LMs store past keys and values in the **KV cache**, instead of recomputing them.

New keys and values at each time step are added to the cache.

Lots of ongoing research: cache compression, smarter memory management, offloading, reusing it across requests, etc.



Another optimization: quantization

During inference, use lower-precision representations of network parameters.

Reduces the space required to store the model.

Precision is the # of bits (binary digits) used to represent a numerical value.

If your model has 65B params:

- float32 (single-precision) → 260 GB
- float16 (half-precision) → 130 GB, w/o sacrificing performance too much!

Remember, we do something like this for **QLoRA** fine-tuning, too!

Motivating an efficient decoding technique

Observation: some tokens are easier to generate than others.

What is the square root of 7?

The square root of 7 is 2.646.

Computing the distribution for each next token is slow for large LMs, and must be done serially:

$$y_t \sim P(\cdot | x, y_1, \dots, y_{t-1})$$

$$y_{t+1} \sim P(\cdot | x, y_1, \dots, y_{t-1}, y_t)$$

... and so on

Speculative decoding

Idea: what if we use a generation from a small LM, e.g. a 1B model, to assist large LM generation?

Let's first approximate, or “guess”, our probability distributions with a faster, smaller LM.

Speculative decoding

First step: quickly sample a draft sequence of length k from a small LM.

An example for $k = 5$:

$$y_1 \sim p(\cdot | x) \quad y_2 \sim p(\cdot | x, y_1) \quad \cdots \quad y_5 \sim p(\cdot | x, y_1, y_2, y_3, y_4)$$

Then, compute the token distribution at each time step with a large LM:

$$q(\cdot | x) \quad q(\cdot | x, y_1) \quad \cdots \quad q(\cdot | x, y_1, y_2, y_3, y_4) \quad q(\cdot | x, y_1, y_2, y_3, y_4, y_5)$$

These q can be computed in a single forward pass! Similar to how we process an entire training example in one pass to compute losses for every time step.

Speculative decoding

Now, let's compare the probability of each token at time step t assigned by the draft model ($\mathbf{p}_t$) and the target model ($\mathbf{q}_t$):

	y_1	y_2	y_3	y_4	y_5
	cats	love	biting	your	toes
$\mathbf{p}_i$	0.8	0.7	0.9	0.8	0.7
$\mathbf{q}_i$	0.9	0.8	0.8	0.3	0.8

For example, this probability corresponds to

$$q_2 = q(\cdot | x, y_1)$$

Speculative decoding

Now, let's compare the probability of each token at time step t assigned by the draft model ($\mathbf{p}_t$) and the target model ($\mathbf{q}_t$):

	y_1	y_2	y_3	y_4	y_5
	cats	love	biting	your	toes
$\mathbf{p}_i$	0.8	0.7	0.9	0.8	0.7
$\mathbf{q}_i$	0.9	0.8	0.8	0.3	0.8

Starting at $t = 1$, we'll decide whether to accept the tokens generated by the draft model.

Speculative decoding

	y_1	y_2	y_3	y_4	y_5
	cats	love	biting	your	toes
p_i	0.8	0.7	0.9	0.8	0.7
q_i	0.9	0.8	0.8	0.3	0.8

Here, $q_i \geq p_i$

This means the target LM likes this token, even more than the draft LM.

→ we accept this token!

Our sequence so far: cats

Speculative decoding

	y_1	y_2	y_3	y_4	y_5
	cats	love	biting	your	toes
p_i	0.8	0.7	0.9	0.8	0.7
q_i	0.9	0.8	0.8	0.3	0.8

Here, $q_i \geq p_i$ again

This means the target LM likes this token, even more than the draft LM.

→ we accept this token!

Our sequence so far: cats love

Speculative decoding

	y_1	y_2	y_3	y_4	y_5
	cats	love	biting	your	toes
p_i	0.8	0.7	0.9	0.8	0.7
q_i	0.9	0.8	0.8	0.3	0.8

Here, $q_i < p_i$

Target LM doesn't like this token as much as the draft LM...

→ we toss a coin with probability q_i / p_i and it lands heads, so we accept it

Our sequence so far: cats love biting

Speculative decoding

	y_1	y_2	y_3	y_4	y_5
	cats	love	biting	your	toes
p_i	0.8	0.7	0.9	0.8	0.7
q_i	0.9	0.8	0.8	0.3	0.8

Here, $q_i < p_i$

Target LM doesn't like this token by a large diff...

→ we toss our q_i / p_i coin again, and given our chances, we end up rejecting it

Our sequence so far: cats love biting

Speculative decoding

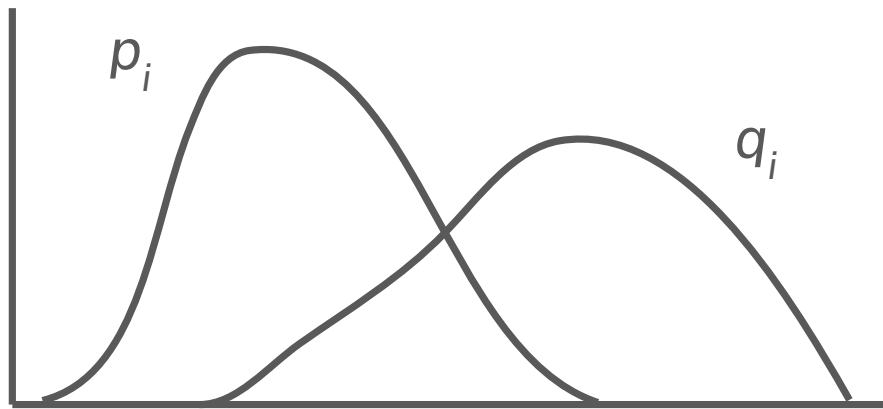
	y_1	y_2	y_3	y_4	y_5
	cats	love	biting	your	toes
p_i	0.8	0.7	0.9	0.8	0.7
q_i	0.9	0.8	0.8	0.3	0.8

Here, $q_i < p_i$

Since we reject the draft LM's token, we sample a new token from our target LM, from the renormalized distribution $(q_i - p_i)_+$

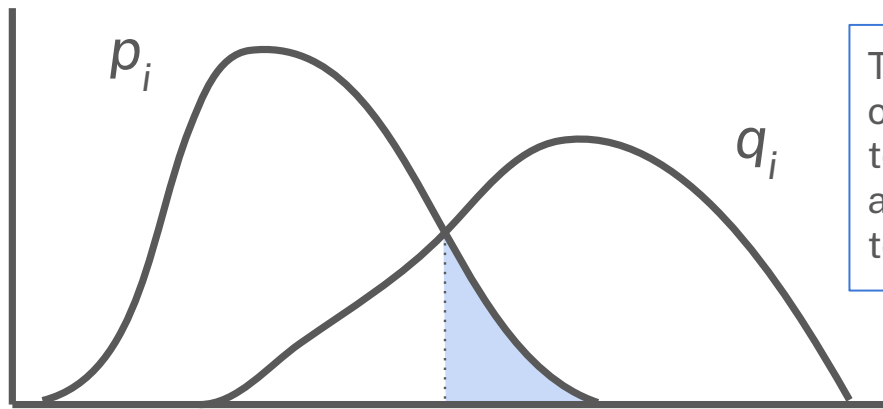
Our sequence so far: cats love biting

Speculative decoding



Why the distribution $(q_i - p_i)_+$?

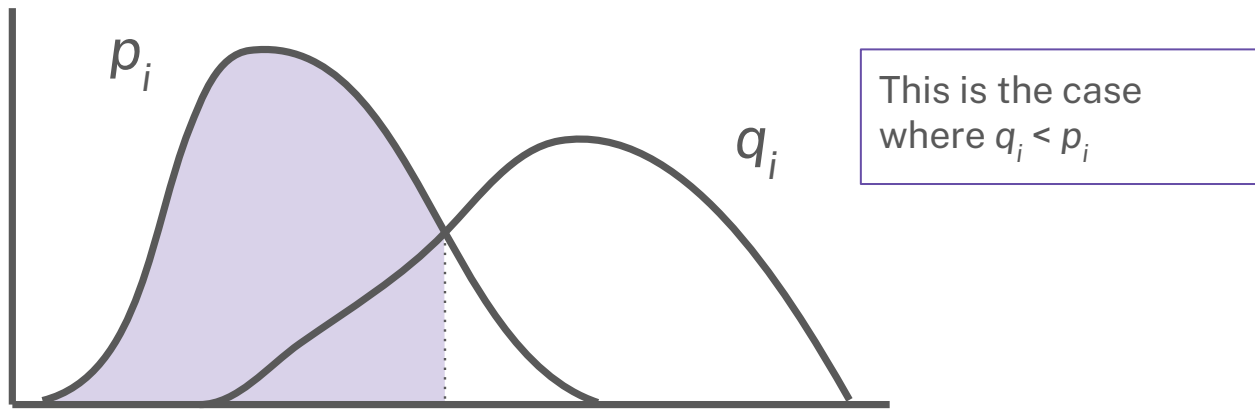
Speculative decoding



This is the case where our draft LM generates a token where $q_i \geq p_i$. We always accept these tokens.

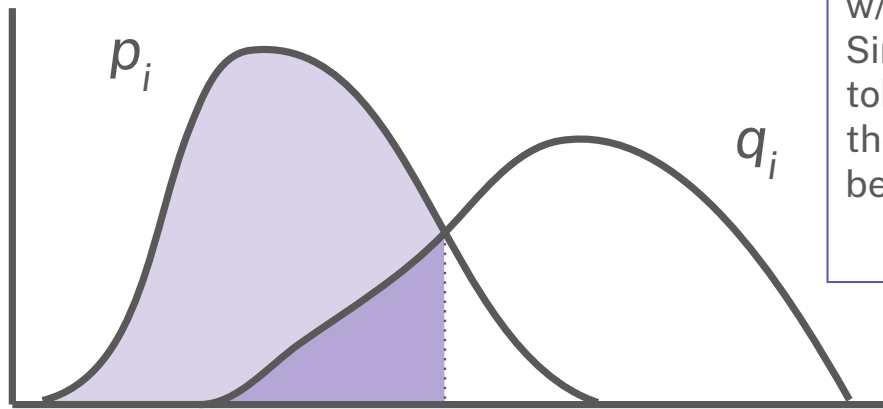
Why the distribution $(q_i - p_i)_+$?

Speculative decoding



Why the distribution $(q_i - p_i)_+$?

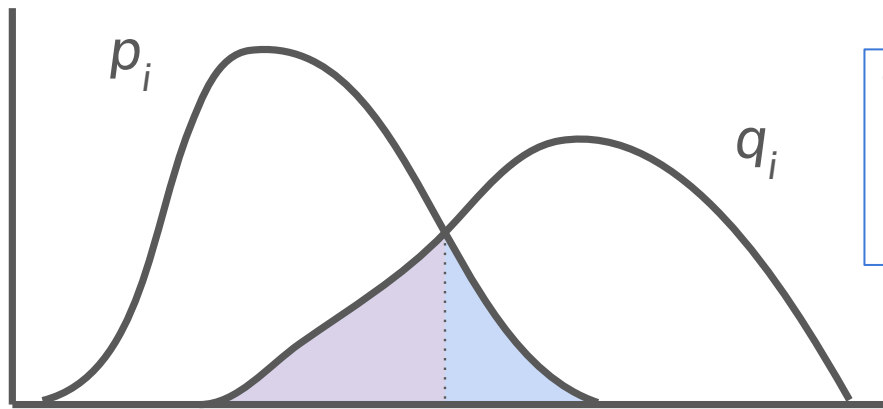
Speculative decoding



We accept cases of $q_i < p_i$
w/ prob q_i / p_i
Since the prob of the
token being drafted is p_i ,
the total prob of the token
being kept overall is
$$p_i \cdot (q_i / p_i) = q_i$$

Why the distribution $(q_i - p_i)_+$?

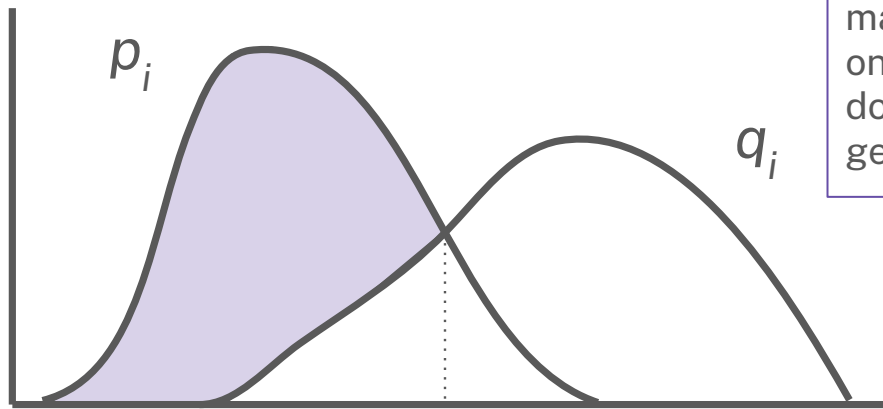
Speculative decoding



This is the total probability we've accepted a drafted token.

Why the distribution $(q_i - p_i)_+$?

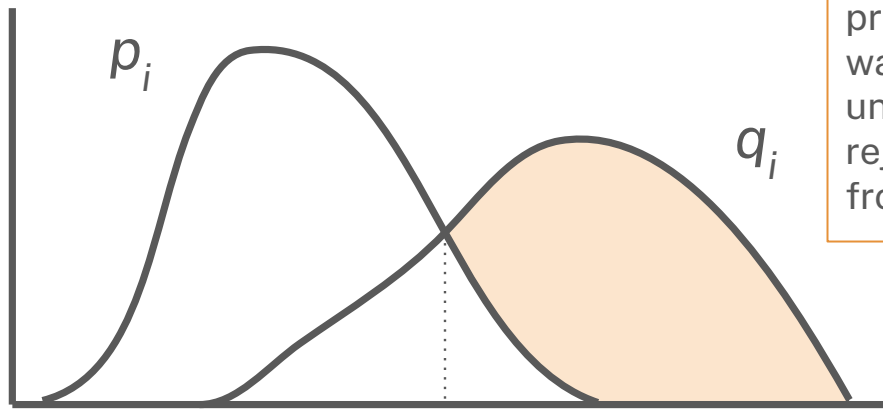
Speculative decoding



So, this is the probability mass the draft LM wastes on tokens the target doesn't like as much, that get rejected.

Why the distribution $(q_i - p_i)_+$?

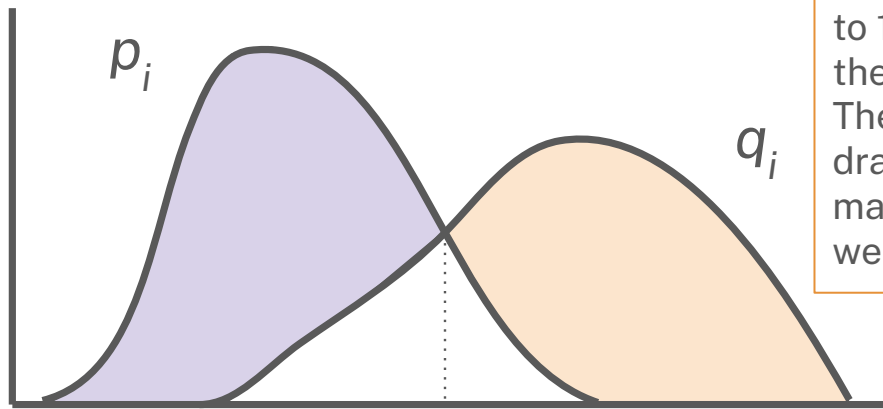
Speculative decoding



This is $(q_i - p_i)_+$ a.k.a. the probability the target LM wants but the draft LM under-predicts. So, after rejection, we resample from this region.

Why the distribution $(q_i - p_i)_+$?

Speculative decoding

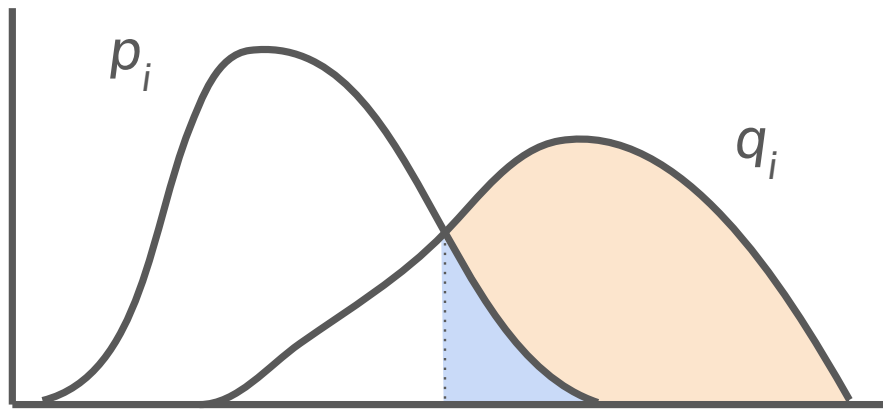


Since both curves are prob distributions that integrate to 1, both of these areas are the same size!

The chance of rejecting a draft (purple) exactly matches the missing mass we resample from (orange).

Why the distribution $(q_i - p_i)_+$?

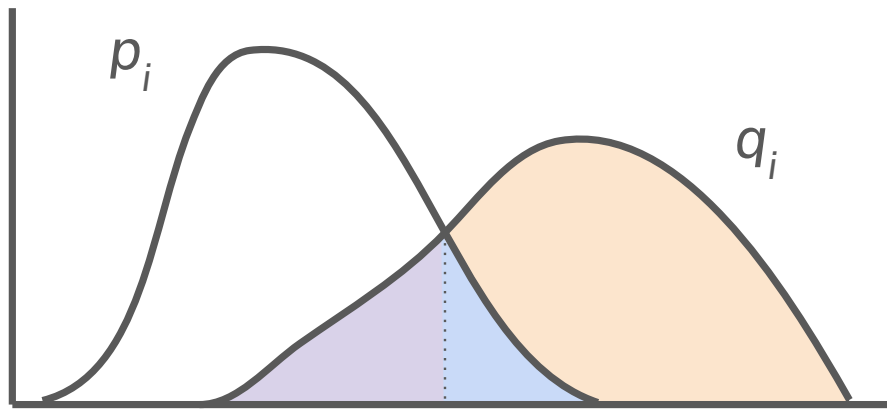
Speculative decoding



The beautiful thing: every token in our final sequence is overall produced with probability q_i .

→ If $q_i \geq p_i$, accepted + resampled = $p_i + (q_i - p_i) = q_i$

Speculative decoding

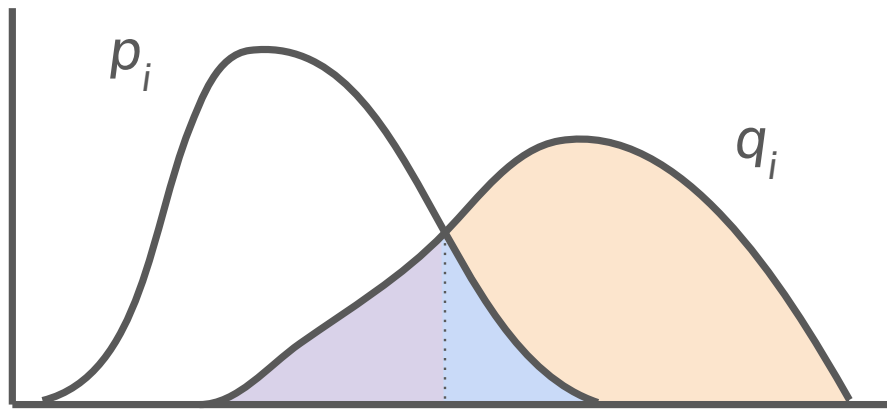


The beautiful thing: every token in our final sequence is overall produced with probability q_i .

→ If $q_i \geq p_i$, accepted + resampled = $p_i + (q_i - p_i) = q_i$

→ If $q_i < p_i$, accepted = q_i

Speculative decoding



The beautiful thing: every token in our final sequence is overall produced with probability q_i . We get the same output in less time!

- If $q_i \geq p_i$, accepted + resampled = $p_i + (q_i - p_i) = q_i$
- If $q_i < p_i$, accepted = q_i

Speculative decoding

	y_1	y_2	y_3	y_4	y_5
	cats	love	biting	your	toes
p_i	0.8	0.7	0.9	0.8	0.7
q_i	0.9	0.8	0.8	0.3	0.8

Okay, we sampled a new token from our target LM, from $(q_i - p_i)_+$.
Now what?

Our sequence so far: cats love biting everyone

Speculative decoding

	y_1	y_2	y_3	y_4	y_5
	cats	love	biting	your 	toes 
p_i	0.8	0.7	0.9	0.8 	0.7 
q_i	0.9	0.8	0.8	0.3	0.8

Everything after the rejected position has to be **thrown away!**

A new round of drafting and comparing starts from the current prefix.

Our sequence so far: cats love biting everyone

Speculative decoding

Speculative sampling is a form of **rejection sampling**

We sample from an easy-to-sample distribution p , produced by a small LM, to reduce the number of passes we must make with our large LM to produce distribution q

With say, a 60M LM (e.g. T5-small) as a draft model and a 11B LM (e.g. T5-XXL) as a target model, we can get 2 to 3 times faster inference!

Limitations: dependence on acceptance rate (e.g. better for highly predictable tasks like code rather than creative writing), and requires extra compute.

Decoding wrap-up

Different decoding algorithms allow us to inject biases that encourage different properties of language generation.

Inference engines (e.g. vLLM, SGLang, LightLLM) implement hardware-optimized inference algorithms for open-weight models.

You should modify sampling hyperparameters for different LLMs and different tasks; usually model cards provide good starting points.

There's still more research in this area to be done! Many connections w/ reasoning and test-time scaling for a future lecture...

Logistics

Upcoming

- Project proposal due *tonight*
- In-class assignment *Wed*
- Mid-course evaluation *Oct 12 - Oct 16*, managed by the department via HelioCampus; check your email for “Please Complete Your UW-Madison Course Evaluation Survey”.
- Rulin’s and Sara’s guest lectures *Oct 12, Oct 14*